AcquirerX Deep-Dive 1/10 — Eureka Server

> **Service:** `eureka-server` **Port:** `8761` **Database:** None **Role:** Service discovery / service registry

---

1. What It Is

Eureka is a **service registry** — essentially a phone directory for microservices. It maintains a live list of all running services, their hostnames, ports, and health status.

In AcquirerX, every business service (auth, merchant, terminal, transaction, risk, settlement, ops, gateway) registers itself with Eureka on startup. When one service needs to call another, it asks Eureka "where is `MERCHANT-SERVICE`?" and Eureka returns the current location.

---

2. Why It Exists

2.1 The problem without service discovery

In a hard-coded URL world, the transaction-service would have a config like:

```
merchant.service.url=http://localhost:9091
```

This breaks the moment:

- A service moves to a different machine in production
- You scale a service to multiple instances on different ports
- A service restarts on a different port
- You deploy to Kubernetes where IPs are ephemeral

2.2 What Eureka solves

- **Decoupled location** — services discover each other by *name*, not URL
- **Multiple instances** — Eureka can return one of several running instances of the same service (load balancing target)
- **Self-healing** — when a service crashes, Eureka removes it after a grace period; when it recovers, it re-registers automatically
- **Centralized registry** — one source of truth for "what is currently running"

2.3 Why it's the first thing to start

Every other service depends on Eureka. If Eureka is down when a service starts, the service will fail to register and inter-service communication will break. Hence the strict startup order:

```
Eureka → Gateway → Auth → Merchant → Terminal → Transaction → Risk → Settlement → Ops → Frontend
```

---

3. The Code

3.1 `EurekaServerApplication.java`

java

```java
package com.acquirerx.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Line-by-line:

- `@SpringBootApplication` — standard Spring Boot bootstrap (combines `@Configuration`, `@EnableAutoConfiguration`, `@ComponentScan`)
- `@EnableEurekaServer` — turns this Spring Boot app into a Eureka registry. Without this annotation, it's just an empty Spring Boot app. With it, Spring Cloud auto-configures the entire registry behaviour.

3.2 `application.properties` (Eureka config)

properties

```
server.port=8761
spring.application.name=eureka-server

# Don't register Eureka with itself
eureka.client.register-with-eureka=false

# Don't fetch the registry (Eureka IS the registry)
eureka.client.fetch-registry=false

# Server URL (for clients to find Eureka)
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

# Disable self-preservation in dev
eureka.server.enable-self-preservation=false
```

Why each setting:

Setting	Reason
`register-with-eureka=false`	Eureka shouldn't register *itself* as a discoverable service — it's the registry
`fetch-registry=false`	Eureka isn't a client, so it doesn't need to fetch the list of services
`service-url.defaultZone`	This is what *clients* use to find Eureka. In dev, all clients point here
`enable-self-preservation=false`	In production, Eureka has a "self-preservation mode" — if too many heartbeats fail at once (e.g., network blip), it assumes the network is bad and stops evicting services. In dev, you want services evicted quickly when they're actually down, so disable it

3.3 `pom.xml` — key dependencies

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

That's it. One dependency. Everything else is auto-configured.

4. How Other Services Register

Every other service in AcquirerX has this dependency:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

And these properties (e.g., merchant-service):

```properties
spring.application.name=merchant-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Plus the annotation:

```java
@SpringBootApplication
@EnableDiscoveryClient   // tells Spring this app should register with discovery
public class MerchantServiceApplication { ... }
```

> Note: in newer Spring Cloud versions, `@EnableDiscoveryClient` is optional — having the eureka-client dependency is enough. AcquirerX includes it for clarity.

---

5. The Registration Flow

```
Service starts up
    ↓
Reads application.properties → finds spring.application.name=merchant-service
    ↓
Reads eureka.client.service-url.defaultZone → http://localhost:8761/eureka/
    ↓
POST to Eureka:
    "I'm MERCHANT-SERVICE, I'm at 192.168.1.x:9091, status=UP"
```

```
    ↓
Eureka adds entry to its in-memory registry
    ↓
Service starts sending heartbeats every 30 seconds
    ↓
If 3 heartbeats missed → Eureka marks instance as DOWN → eventually evicts it
```

---

6. The Discovery Flow (How Services Find Each Other)

In AcquirerX, services find each other through **Feign clients** (declarative HTTP clients). Example from transaction-service:

```java
@FeignClient(name = "MERCHANT-SERVICE", fallback = MerchantServiceClientFallback.cla
public interface MerchantServiceClient {
    @GetMapping("/api/v1/merchants/{id}")
    Map<String, Object> getMerchantById(@PathVariable Long id);
}
```

What happens at runtime:

1. Transaction-service calls `merchantClient.getMerchantById(1L)`
2. Feign sees `name = "MERCHANT-SERVICE"` (no URL)
3. Feign asks Eureka: "where is MERCHANT-SERVICE?"
4. Eureka returns: "It's at `192.168.1.x:9091`"
5. Feign sends `GET http://192.168.1.x:9091/api/v1/merchants/1`
6. Response flows back through Feign as `Map<String, Object>`

The transaction-service code never hard-codes the merchant URL. That's the whole point.

---

7. The Eureka Dashboard

Eureka exposes a built-in HTML dashboard at `http://localhost:8761`:

```
Application                AMIs     Availability Zones     Status
-------------------------------------------------------------------
AUTH-SERVICE               n/a      (1)                    UP (1) - host:9099
MERCHANT-SERVICE           n/a      (1)                    UP (1) - host:9091
TERMINAL-SERVICE           n/a      (1)                    UP (1) - host:9092
TRANSACTION-SERVICE        n/a      (1)                    UP (1) - host:9093
RISK-SERVICE               n/a      (1)                    UP (1) - host:9094
SETTLEMENT-SERVICE         n/a      (1)                    UP (1) - host:9095
OPS-SERVICE                n/a      (1)                    UP (1) - host:9096
API-GATEWAY                n/a      (1)                    UP (1) - host:9090
```

You should see exactly 8 services UP when everything is running. If any are missing, that service either failed to start or can't reach Eureka.

---

8. Heartbeat & Eviction

Event	Default timing
Service sends heartbeat	every 30 seconds
Eureka evicts instance	after 90 seconds of missed heartbeats
Self-preservation kicks in (prod only)	if >85% of heartbeats fail in a renewal window

In AcquirerX dev, self-preservation is disabled, so a stopped service disappears from Eureka within ~90 seconds.

---

9. Why Service Names Are Uppercase

You'll notice Eureka shows `MERCHANT-SERVICE` (uppercase) but the property is `spring.application.name=merchant-service` (lowercase). Eureka uppercases all service names internally. When Feign or Gateway resolves a service name, it's case-insensitive — both `MERCHANT-SERVICE` and `merchant-service` work.

In the gateway routes, you see this pattern:

```properties
spring.cloud.gateway.routes[2].uri=lb://MERCHANT-SERVICE
```

The `lb://` prefix means **load-balanced through Eureka.** The gateway looks up `MERCHANT-SERVICE` in Eureka, picks an instance (using round-robin if there are multiple), and forwards the request.

---

10. Failure Scenarios

Scenario	What happens
Eureka is down when other services start	Services fail to register; inter-service calls fail. Cannot recover until Eureka starts.
Eureka goes down after services have registered	Services keep last-known registry cache. Existing calls work. New registrations fail.
A service crashes	Eureka evicts it after ~90s. Feign calls to it fail (handled by Resilience4J fallbacks).
A service restarts on a different port	Re-registers automatically. Old entry evicted. New calls go to new port.

---

11. Health Check Endpoints

Every service registered with Eureka exposes:

```
GET http://<service>:port/actuator/health
```

Eureka uses this to determine if the service is UP, DOWN, or OUT_OF_SERVICE. Spring Boot Actuator handles this automatically via the `eureka-client` dependency.

---

12. Why AcquirerX Uses Eureka (vs Alternatives)

Option	Why not chosen
Hard-coded URLs	Breaks at scale, doesn't support multiple instances or restarts on different ports

Option	Why not chosen
DNS-based discovery (Kubernetes)	Requires Kubernetes — adds complexity for a learning project
Consul, Zookeeper	More feature-rich but adds operational complexity
Eureka	Lightweight, Spring-native, perfect fit for Spring Cloud ecosystem, single dependency

---

13. Summary

Aspect	Detail
Purpose	Live registry of running services
Port	8761
Dependencies	`spring-cloud-starter-netflix-eureka-server`
Annotations	`@SpringBootApplication`, `@EnableEurekaServer`
Database	None (in-memory)
Started by	Always first in startup sequence
Used by	Every other service (registers + discovers via Eureka)
Dashboard	`http://localhost:8761`
Failure impact	New service registrations break; existing connections continue using cached registry

---

Next: API Gateway — routing, JWT validation, rate limiting, CORS.